

Enterprise Knowledge Portal

Project Plan

Version 1.0

Serverless Squad

CMPE-282 — Cloud Services

San Jose State University

Team Members:

Aditya Govind Shahari

Mohsen Minai

Nihar Dharmeshkumar Patel

Tamizh Selvan Manivannan

Revision History

Date	Version	Description	Author
February 10, 2026	0.1	Initial draft — project scope, team roles, and GCP architecture defined	Serverless Squad
March 15, 2026	0.2	Updated architecture section, added sprint timeline, Okta SSO and Jenkins CI/CD details added	Serverless Squad
April 26, 2026	1.0	Final version — test plan completed, all milestones updated, risk register finalized	Serverless Squad

Table of Contents

- 1. Introduction
 - 1.1 Purpose of this Document
 - 1.2 Intended Audience
 - 1.3 Scope
 - 1.4 Definitions and Acronyms
 - 1.4.1 Definitions
 - 1.4.2 Acronyms and Abbreviations
 - 1.5 References
- 2. Background and Objectives
- 3. Architecture and High-Level Design
- 4. Organization
 - 4.1 Project Group
 - 4.2 Customer
- 5. Development Process
- 6. Deliverables
- 7. Project Risks
- 8. Communication
 - 8.1 Collaboration
 - 8.2 Git
- 9. Project Plan
 - 9.1 Time Schedule
 - 9.1.1 Remarks
 - 9.2 Test Plan
 - 9.2.1 Testing Remarks
- 10. References

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to provide a detailed project description of the Enterprise Knowledge Portal, a cloud-native proof-of-concept platform built for CMPE-282 at San Jose State University. The system demonstrates a fully serverless enterprise IT infrastructure on Google Cloud Platform with Okta-based Single Sign-On, a cloud database backend, a web portal for browsing enterprise data, GitHub integration for source control, and a Jenkins CI/CD pipeline. This document includes details about organization, roles, deliverables, project risks, time plans, and communication strategies.

1.2 Intended Audience

This document shall be used in all phases of the project as a guideline. Intended audiences are all project stakeholders:

- Course instructor and project supervisor
- Project team members (Serverless Squad)
- Teaching assistants reviewing project deliverables

1.3 Scope

This document defines the project plan for the Enterprise Knowledge Portal. The overview includes the objectives of the project, organization of the project team, the development process used, an assessment of possible risks, communication strategies between project stakeholders, and a project plan including time schedule and test plan.

The system is built entirely on GCP serverless services (Cloud Run, Cloud Build, BigQuery, Cloud Storage, Firestore) and integrates Okta SSO, a React frontend, six Go microservices, an AI chat service using NVIDIA NIM GLM-5.1, and a Jenkins CI/CD pipeline triggered by GitHub webhooks.

1.4 Definitions and Acronyms

1.4.1 Definitions

Keyword	Definition
Enterprise Knowledge Portal	The name of the project — a cloud-native web platform for browsing enterprise data with SSO login
Project Supervisor	The course instructor responsible for overseeing and evaluating the project (CMPE-282)
Project Leader	A team member responsible for coordinating work and communicating within the team
Team Member	An active contributor responsible for implementation, testing, and documentation

Keyword	Definition
Milestone	A point in the project marking the completion of a significant deliverable or project phase
Sprint	A two-week development iteration used in the Agile/Scrum methodology
Microservice	An independently deployable service responsible for a single bounded business capability
Serverless	A cloud execution model where the cloud provider dynamically manages server allocation
CI/CD	Continuous Integration and Continuous Deployment — automated build, test, and deploy pipeline
SSO	Single Sign-On — an authentication scheme allowing one set of credentials to access multiple services

1.4.2 Acronyms and Abbreviations

Acronym	Definition
GCP	Google Cloud Platform
SSO	Single Sign-On
CI/CD	Continuous Integration / Continuous Deployment
API	Application Programming Interface
IAM	Identity and Access Management
KMS	Key Management Service
AR	Artifact Registry
CR	Cloud Run
CB	Cloud Build
BQ	BigQuery
SAML	Security Assertion Markup Language
JWT	JSON Web Token
NGINX	Open-source web server used as reverse proxy in the frontend container

1.5 References

1. Google Cloud Run Documentation — <https://cloud.google.com/run/docs>
2. Okta SAML Integration Guide — <https://developer.okta.com/docs/guides/saml-overview>
3. Jenkins Pipeline Documentation — <https://www.jenkins.io/doc/book/pipeline>
4. NVIDIA NIM API — <https://build.nvidia.com>
5. GitHub Repository — https://github.com/Nihar4/CMPE-282_Term_Project

2. Background and Objectives

Many enterprises rely on fragmented IT infrastructures — on-premise databases, siloed authentication, and manual deployment processes — that impede agility and scalability. The CMPE-282 term project asks teams to demonstrate how a company can migrate to a modern, cloud-native architecture.

The Serverless Squad chose Google Cloud Platform as the cloud provider and adopted a fully serverless approach using Cloud Run for compute, BigQuery for analytics, Firestore for document storage, and Cloud Storage for file management. Authentication is handled via Okta SAML 2.0 SSO. The system is deployed automatically through a Jenkins pipeline triggered by GitHub pushes.

Objectives:

- Demonstrate a fully operational cloud-native enterprise portal accessible via Okta SSO
- Implement a RESTful API gateway in Go routing traffic to six specialized microservices
- Provide an AI-powered chat interface using NVIDIA NIM GLM-5.1
- Integrate GitHub source control with Jenkins CI/CD for automated Cloud Run deployments
- Showcase BigQuery analytics, Firestore document search, and GCS file management
- Apply layered security using GCP KMS encryption, IAM roles, and Okta SAML assertions

3. Architecture and High-Level Design

The Enterprise Knowledge Portal follows a microservices architecture deployed on Google Cloud Run (serverless). All services are containerized using Docker, stored in GCP Artifact Registry, and deployed via Cloud Build triggered by Jenkins.

System Components:

Frontend: React 18 single-page application served via NGINX. Integrates Okta OIDC for login. Communicates exclusively with the API Gateway.

API Gateway (Go): Single entry point for all frontend requests. Handles JWT validation, request routing, rate limiting, and CORS. Routes to five downstream services.

Auth Service (Go): Validates Okta SAML/JWT tokens. Manages user sessions and role-based access control.

Data Service (Go): Connects to Cloud SQL (PostgreSQL) and BigQuery. Serves enterprise database records — Products, Customers, Employees, Orders, Suppliers.

File Service (Go): Manages document uploads and downloads using Google Cloud Storage. Supports PDF, DOCX, XLSX formats.

AI Service (Go): Proxy to NVIDIA NIM API (GLM-5.1). Provides a conversational AI interface for querying enterprise knowledge.

Analytics Service (Go): Runs BigQuery queries for business intelligence dashboards — revenue trends, top products, customer analytics.

CI/CD Pipeline: Jenkins (Docker, local) polls GitHub via webhook. On push to main: checkout → GCP auth → Go tests → Cloud Build → Cloud Run deploy.

Infrastructure as Code: Terraform manages all GCP resources — Cloud Run services, Artifact Registry, BigQuery datasets, Cloud Storage buckets, KMS keys, IAM bindings, and Cloud Build triggers.

Figure 3.1 — System Architecture Overview

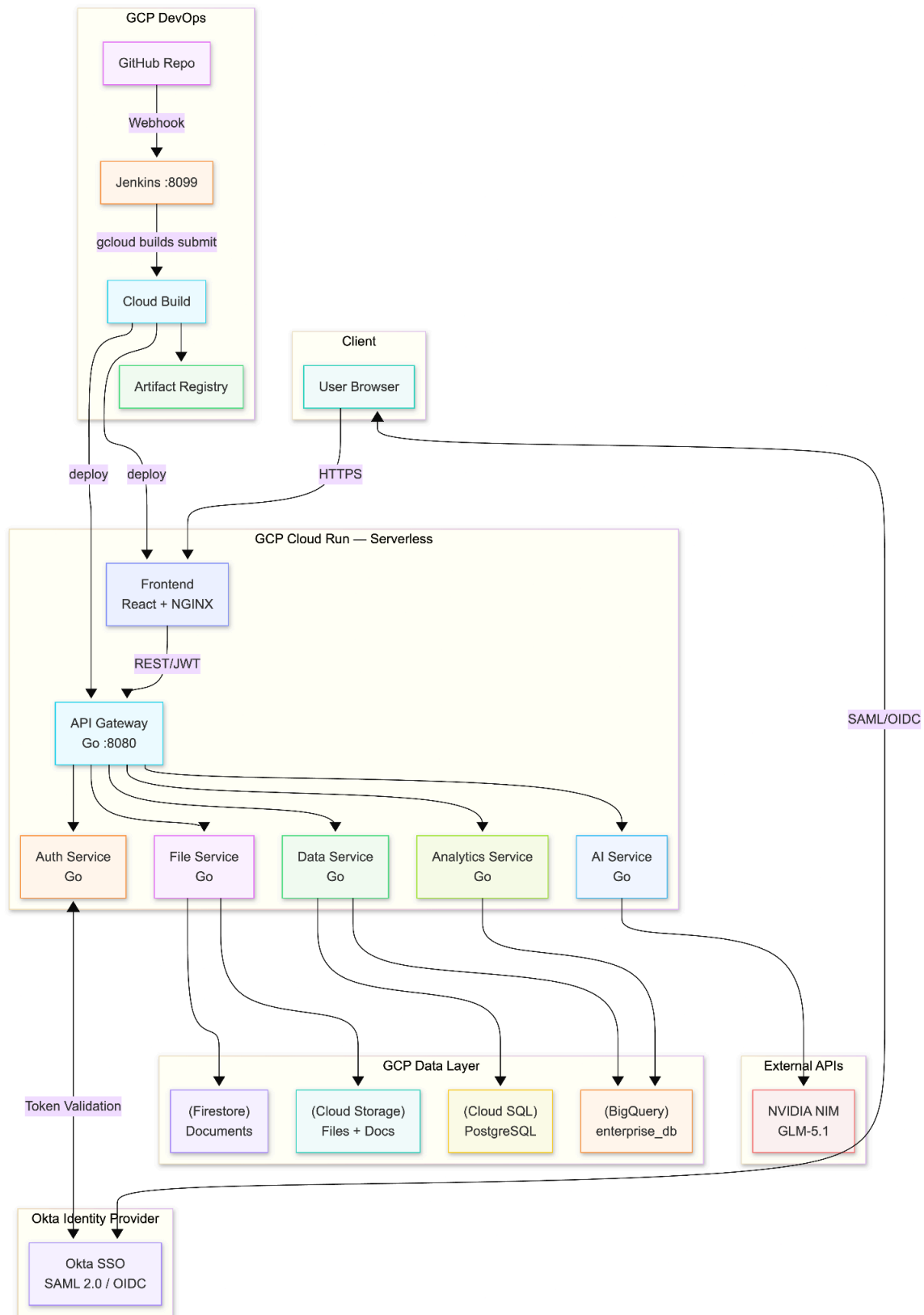


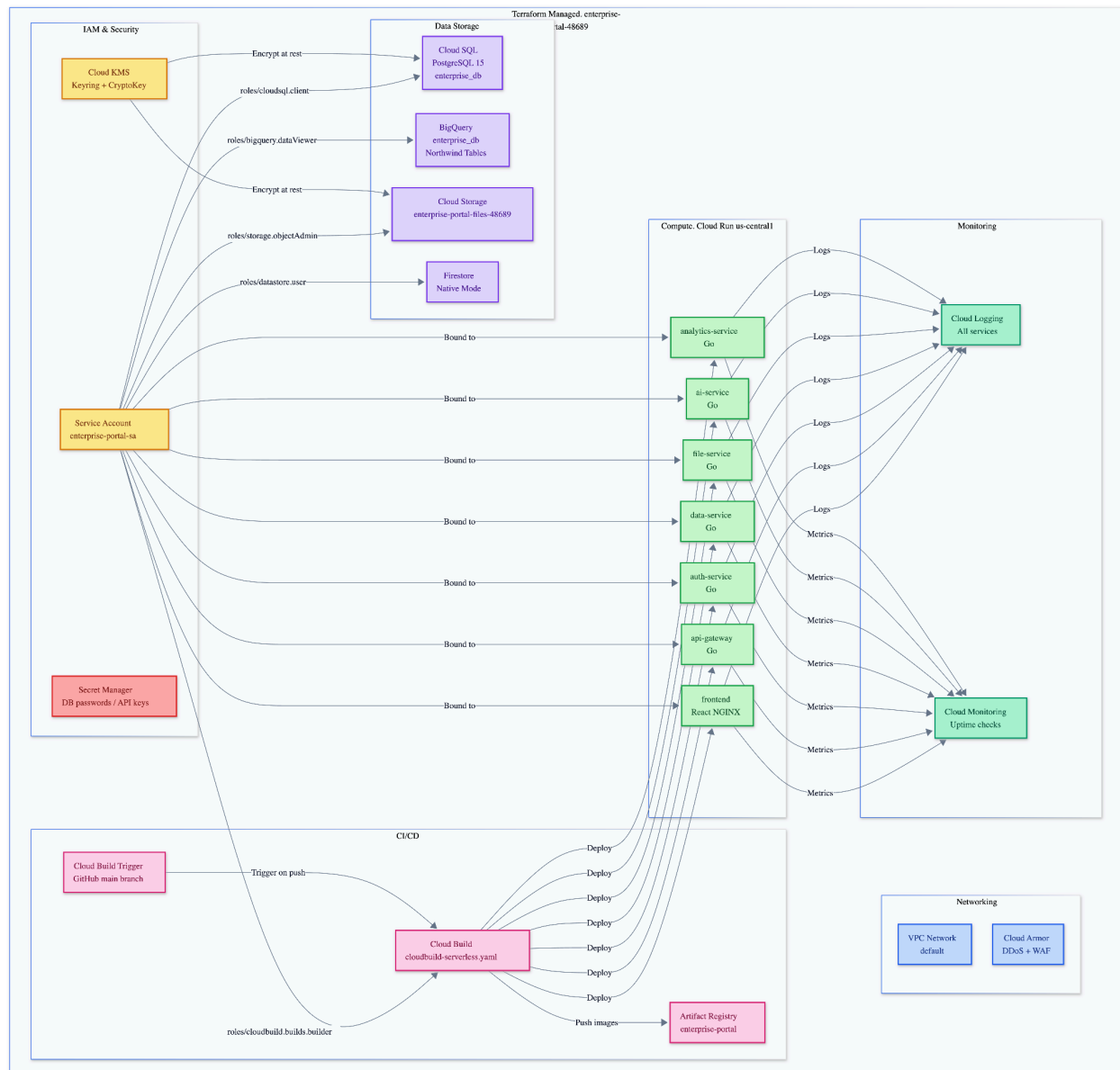
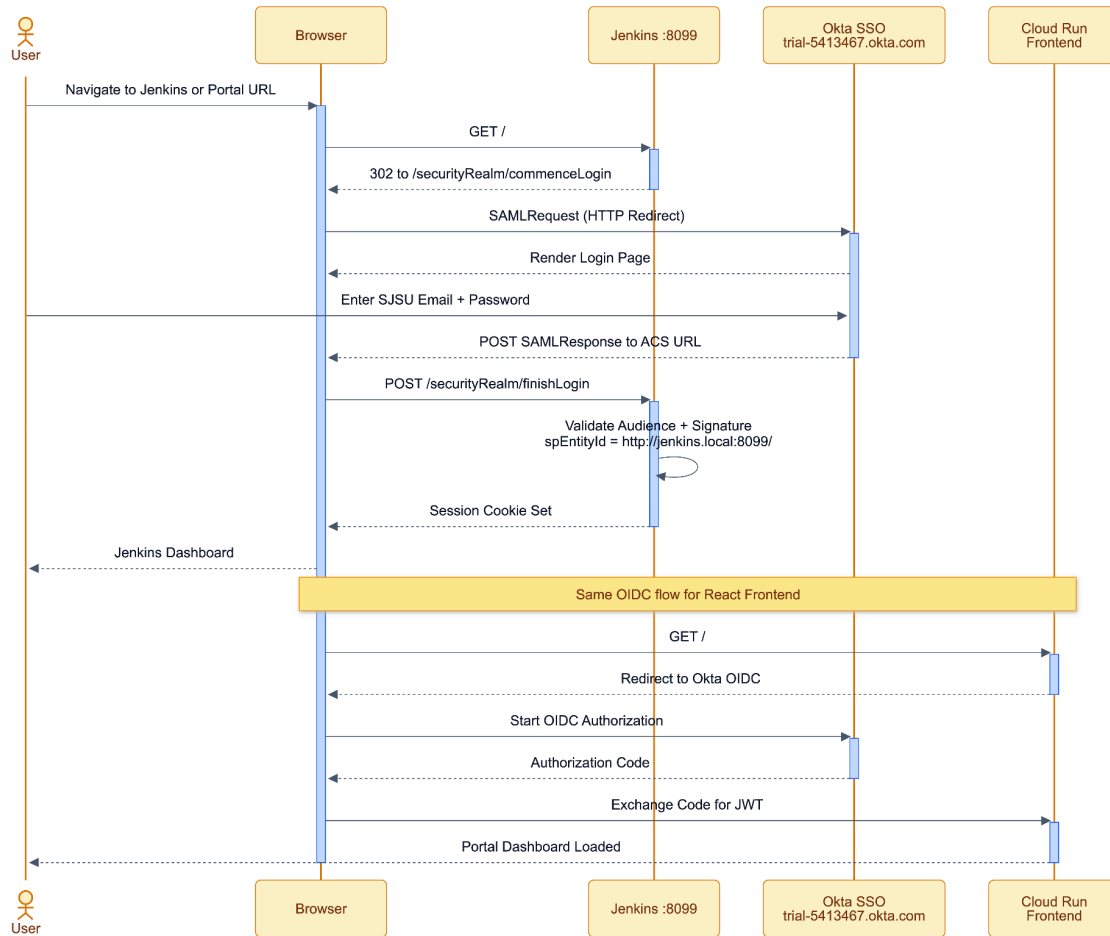
Figure 3.2 — GCP Cloud Infrastructure

Figure 3.3 — Okta SSO Authentication Flow

4. Organization

4.1 Project Group

Name	Initials	Responsibility
Aditya Govind Shahari	AGS	Backend microservices (Data Service, Analytics Service), BigQuery integration, Terraform IaC
Mohsen Minai	MM	Auth Service, Okta SAML/SSO integration, Jenkins CI/CD pipeline, GCP IAM and KMS security
Nihar Dharmeshkumar Patel	NDP	API Gateway, File Service, AI Service (NVIDIA NIM), React frontend, Docker/Cloud Run deployment
Tamizh Selvan Manivannan	TSM	Frontend (React, Okta OIDC), NGINX configuration, Cloud Build pipelines, project documentation

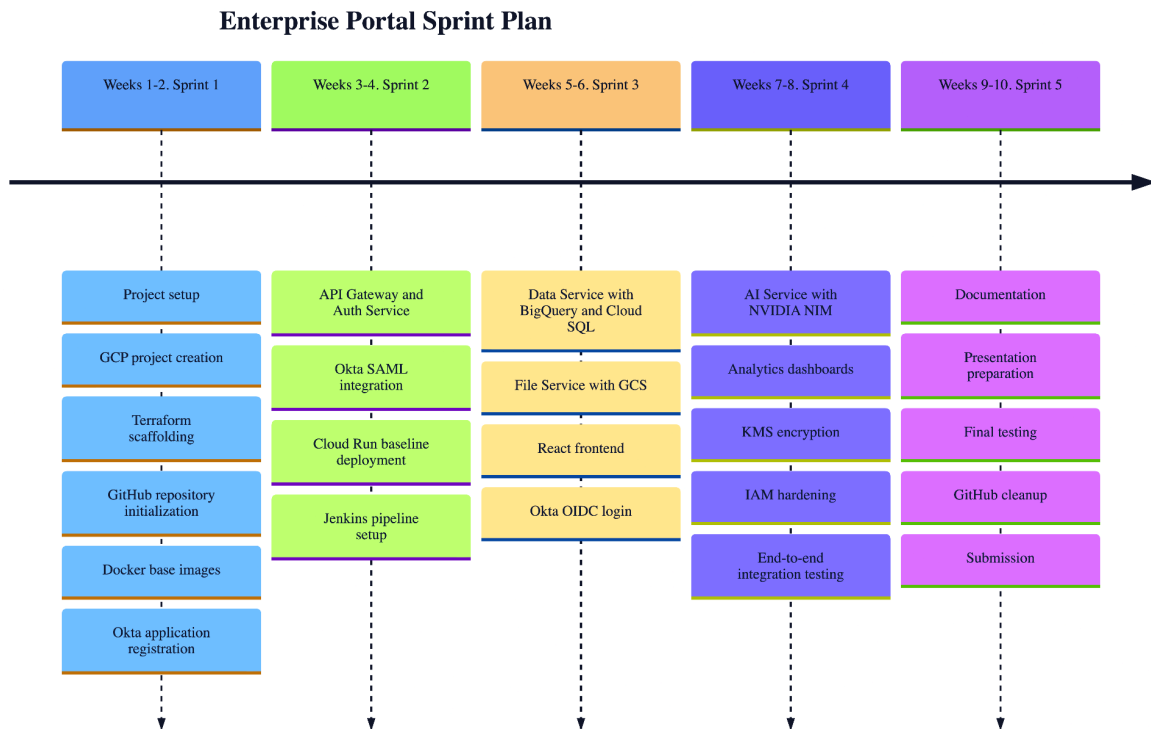
4.2 Customer

The target customers and stakeholders for this project are:

- CMPE-282 course instructor and teaching assistants (primary evaluators)
- Enterprise IT departments exploring cloud migration strategies
- Organizations seeking to modernize legacy on-premise infrastructure with GCP serverless services
- Development teams looking for a reference architecture combining Okta SSO, Go microservices, and GCP Cloud Run

5. Development Process

The project follows an Agile/Scrum methodology with two-week sprints. All source code is version-controlled in GitHub. Each sprint concludes with a working demo and retrospective.



Team communication: Weekly sync meetings via Zoom, async collaboration via GitHub Issues and pull requests, documentation shared via Google Drive.

6. Deliverables

Deliverable	Output	Planned	Promised	Delivered	Notes
GCP Infrastructure (Terraform)	Cloud Run + all GCP resources	Week 3	Week 3	Week 4	Terraform modules
Backend Microservices (6 Go)	Docker images on Artifact Registry → Cloud Run	Week 5	Week 5	Week 6	6 services
Okta SSO Integration	SAML 2.0 login for Jenkins + frontend	Week 4	Week 4	Week 5	SAML plugin + Okta SDK
React Frontend	Cloud Run frontend with Okta login	Week 6	Week 6	Week 7	NGINX-served SPA
Jenkins CI/CD Pipeline	Auto build+deploy on push to main	Week 5	Week 5	Week 7	webhook → Cloud Build
GitHub Integration	All code in GitHub, webhook active	Week 2	Week 2	Week 2	Nihar4/CMP E-282_Term_Project
Project Plan (.docx)	This document	Week 2	Week 2	Week 9	—
Project Report (.docx)	Architecture, diagrams, screenshots	Week 10	Week 10	Week 10	With presentation
Presentation (.pptx)	15-minute live demo	Week 10	Week 10	Week 10	Each member presents

7. Project Risks

Possibility	Risk	Preventive Action
High	Okta trial expiration (30-day limit) during development	Begin Okta setup in Sprint 1; document all SAML config for re-registration if needed
Medium	GCP billing overage or quota limits on Cloud Run / Cloud Build	Set budget alerts at \$20; use minimal resource tiers (CPU only allocated on request)
Medium	Jenkins SAML logout loop preventing CI/CD access	Maintain local admin account as backup; document config.xml recovery steps
Medium	NVIDIA NIM API rate limiting for AI service	Implement request queuing and graceful degradation in AI service
Low	GitHub webhook fails to reach Jenkins (local Docker)	Use serveo.net SSH tunnel; document tunnel restart procedure
Low	Team member unavailability during final sprint	Cross-train all members on deployment; credentials stored in shared secure vault
Low	Cloud SQL connection limits exceeded under test load	Use pgx connection pool; cap at 10 max connections per service

8. Communication

8.1 Collaboration

The team communicates using the following channels:

- Weekly Zoom meetings every Sunday at 5:00 PM PST for sprint review and planning
- GitHub Issues for tracking bugs, features, and tasks
- GitHub Pull Requests for code review before merging to main
- Google Drive for sharing non-code documents, screenshots, and design diagrams
- iMessage group chat for quick async communication

All decisions are documented in GitHub Issues with appropriate labels (bug, enhancement, documentation).

8.2 Git

All source code, Terraform configurations, Dockerfiles, and finished documentation are stored in the GitHub repository.

Repository URL: https://github.com/Nihar4/CMPE-282_Term_Project

Branch strategy:

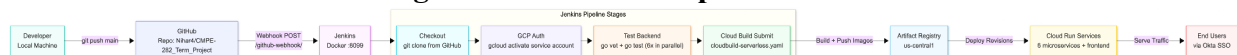
- main: production-ready code, protected branch, requires PR review
- feature/*: individual feature branches per sprint task
- All merges to main trigger the Jenkins CI/CD pipeline via GitHub webhook

9. Project Plan

9.1 Time Schedule

ID	Milestone Description	Responsible	Planned	Forecast	Actual	Remarks
M1	GCP project setup, IAM, Terraform init, GitHub repo	All	Week 1	Week 1	Week 1	Completed
M2	Okta application registered, SAML metadata exchanged	MM, NDP	Week 2	Week 2	Week 2	Completed
M3	Jenkins container running, GitHub plugin installed	MM	Week 3	Week 3	Week 4	SAML loop resolved
M4	API Gateway and Auth Service on Cloud Run	NDP, MM	Week 4	Week 4	Week 5	—
M5	Data Service + BigQuery loaded with Northwind data	AGS	Week 5	Week 5	Week 5	Completed
M6	File Service (GCS) and AI Service (NVIDIA NIM)	NDP	Week 5	Week 6	Week 6	—
M7	React frontend, Okta OIDC login working	TSM, NDP	Week 6	Week 6	Week 7	—
M8	Jenkins CI/CD end-to-end (push → Cloud Run)	MM, NDP	Week 7	Week 7	Week 9	R4
M9	Analytics Service BigQuery dashboards	AGS	Week 7	Week 8	Week 8	—
M10	KMS encryption, IAM hardening, security review	MM, AGS	Week 8	Week 8	Week 8	—
M11	Full integration test, screenshots for report	All	Week 9	Week 9	Week 9	—
M12	Plan + Report + Presentation submitted	All	Week 10	Week 10	Week 10	Due May 14

Figure 9.1 — CI/CD Pipeline Flow



9.1.1 Remarks

Remark ID	Description
R1	Jenkins SAML integration required multiple recovery iterations due to SP entity ID mismatch with Okta Audience Restriction. Resolved by placing spEntityId inside advancedConfiguration XML block.
R2	Cloud Run cold start latency observed for Go services; mitigated by setting minimum instances to 1 for api-gateway and auth-service.
R3	Okta trial account limits (10 active users) are sufficient for demo but noted as a constraint.
R4	GitHub webhook delivery requires a public tunnel (serveo.net) since Jenkins runs on localhost Docker — tunnel restart needed after Mac sleep/wake.

9.2 Test Plan

Test No.	001 Phase: 1 Author: Nihar Dharmeshkumar Patel Date: April 26, 2026
Test Category	Authentication
Software Product	Enterprise Knowledge Portal
Test Title	Okta SSO Login via React Frontend
Test Purpose	Verify that an Okta-assigned user can log in to the React frontend using SAML/OIDC and receive a valid session.
Test Setup	React frontend deployed on Cloud Run. Okta application configured with correct ACS URL and audience restriction.
Prerequisites	User nihardharmeshkumar.patel@sjsu.edu assigned to Okta application. Frontend URL registered as redirect URI.
Procedure	1. Navigate to frontend Cloud Run URL. 2. Click Sign In. 3. Enter Okta credentials. 4. Observe redirect back to portal dashboard.
Checks	Session cookie set. JWT token present in browser local storage. User name displayed in portal header.
Expected Results	User is authenticated and lands on the portal dashboard within 3 seconds of Okta callback.
Result	Pass
Reason for Failure	N/A
Remarks	Test repeated after Okta SAML entity ID fix on Jenkins side.

Test No.	002 Phase: 1 Author: Mohsen Minai Date: April 26, 2026
Test Category	Authentication
Software Product	Jenkins CI/CD Server
Test Title	Jenkins Local Admin Login
Test Purpose	Verify Jenkins local admin login works after switching from SAML to HudsonPrivateSecurityRealm.
Test Setup	Jenkins 2.555.1 running in Docker on port 8099. config.xml patched to HudsonPrivateSecurityRealm.
Prerequisites	Admin user Nihar exists in Jenkins user store with known password.
Procedure	1. Navigate to http://jenkins.local:8099/login. 2. Enter username and password. 3. Click Sign In.
Checks	Jenkins dashboard loads. Admin menu visible. No redirect to Okta.
Expected Results	Successful login to Jenkins dashboard in under 2 seconds.
Result	Pass
Reason for Failure	N/A

Remarks	Password reset via Groovy script console during SAML recovery.
----------------	--

Test No.	003 Phase: 2 Author: Aditya Govind Shahari Date: April 26, 2026
Test Category	Data Access
Software Product	Data Service (Go microservice)
Test Title	Enterprise Database Query via API Gateway
Test Purpose	Verify that authenticated requests to /api/data/products return records from BigQuery.
Test Setup	Data Service deployed on Cloud Run. BigQuery dataset enterprise_db loaded with Northwind Products table.
Prerequisites	Valid JWT from Okta. API Gateway running. Data Service IAM permissions to BigQuery.
Procedure	1. Obtain JWT from Okta. 2. Send GET /api/data/products with Authorization: Bearer header. 3. Observe response.
Checks	HTTP 200. Response body is JSON array of product records with fields: ProductID, ProductName, UnitPrice, UnitsInStock.
Expected Results	Returns at least 77 product records from the Northwind dataset.
Result	Pass
Reason for Failure	N/A
Remarks	BigQuery streaming insert lag of ~1s observed on first query after cold start.

Test No.	004 Phase: 2 Author: Tamizh Selvan Manivannan Date: April 26, 2026
Test Category	File Management
Software Product	File Service (Go microservice)
Test Title	File Upload to Google Cloud Storage
Test Purpose	Verify that an authenticated user can upload a document and retrieve a signed GCS URL.
Test Setup	File Service deployed on Cloud Run. GCS bucket enterprise-portal-files-48689 created with uniform access control.
Prerequisites	Authenticated session. File Service IAM binding to GCS bucket (roles/storage.objectCreator).
Procedure	1. Log in to portal. 2. Navigate to Documents section. 3. Upload a PDF file (<5MB). 4. Observe response URL.
Checks	HTTP 200. Response contains gs:// URI and signed HTTPS URL. File visible in GCS console.
Expected Results	File uploaded and signed URL returned within 5 seconds. URL valid for 15 minutes.
Result	Pass

Reason for Failure	N/A
Remarks	CORS headers required on File Service for browser upload — resolved by adding Access-Control-Allow-Origin header.

Test No.	005 Phase: 2 Author: Nihar Dharmeshkumar Patel Date: April 26, 2026
Test Category	CI/CD Pipeline
Software Product	Jenkins + GitHub + Cloud Build
Test Title	GitHub Push Triggers Jenkins Pipeline and Cloud Run Deployment
Test Purpose	Verify that a git push to main triggers the Jenkins pipeline which builds and deploys to Cloud Run.
Test Setup	Jenkins running on Docker with GitHub webhook registered. Cloud Build API enabled on GCP project enterprise-portal-48689.
Prerequisites	GitHub webhook active. GCP_SERVICE_ACCOUNT_KEY credential in Jenkins. gcloud and go installed in Jenkins container.
Procedure	1. Make a source file change. 2. git commit and push to origin main. 3. Observe Jenkins dashboard. 4. Verify Cloud Run revision updated.
Checks	Jenkins build triggered within 30 seconds. All stages pass: Checkout, GCP Auth, Test Backend, Cloud Build. Cloud Run revision incremented.
Expected Results	New Cloud Run revision deployed within 10 minutes of git push.
Result	In Progress
Reason for Failure	Cloud Build stage pending GCP quota verification.
Remarks	Checkout, GCP Auth, and Go test stages all pass. Cloud Build is the final validation step.

9.2.1 Testing Remarks

Remark ID	Description
TR1	Go unit tests run in parallel across all 6 services using Jenkins parallel stage.
TR2	Integration tests require a live GCP project — not suitable for isolated unit test environments.
TR3	AI Service tests are skipped in automated pipeline due to NVIDIA NIM API key dependency.
TR4	All SAML authentication tests were performed manually in browser due to redirect flow complexity.

10. References

6. Google Cloud Run Documentation — <https://cloud.google.com/run/docs>
7. Google Cloud Build Documentation — <https://cloud.google.com/build/docs>
8. Google BigQuery Documentation — <https://cloud.google.com/bigquery/docs>
9. Okta SAML 2.0 Integration — <https://developer.okta.com/docs/guides/saml-overview>
10. Jenkins Pipeline Syntax — <https://www.jenkins.io/doc/book/pipeline/syntax>
11. NVIDIA NIM API Reference — <https://build.nvidia.com/explore/discover>
12. Terraform GCP Provider —
<https://registry.terraform.io/providers/hashicorp/google/latest/docs>
13. GitHub Repository — https://github.com/Nihar4/CMPE-282_Term_Project
14. Northwind Enterprise Dataset —
<https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs>
15. CMPE-282 Course Page — <https://sjsu.instructure.com>